



MIDTOWN MOTORS USED CARS APPLICATION

CS 6400 Phase 2 Report – Team 15

Ugochukwu Onuoha (uonuoha3)
Mattis Voss (mvoss32)
Monroe Stephenson (mstephenson31)
Andre Dingiswayo (adingiswayo3)
Jose Antonio Rodriguez Albala (jalbala3)

Table of Contents	1
1 Abstract Code + SQL.....	2
1.1 Search Vehicles.....	2
1.2 Authenticate User.....	19
1.3 View Vehicle Details	20
1.4 Modify Part Status.....	25
1.5 Confirm Sale of Vehicle	26
1.6 Lookup/Add Customer	27
1.7 Add Vehicle.....	30
1.8 Lookup/Add Vendor	32
1.9 Add Part to Order	33
1.10 Generate Seller History Report	35
1.11 Generate Avg Time in Inventory Report	36
1.12 Generate Price Per Condition Report	38
1.13 Generate Parts Statistics Report.....	39
1.14 Generate Monthly Sales/Drilldown Report.....	40

Table of Contents

Table of Contents	1
1 Abstract Code + SQL.....	3
1.1 Search Vehicles.....	3
1.2 Authenticate User.....	20
1.3 View Vehicle Details	21
1.4 Modify Part Status.....	26
1.5 Confirm Sale of Vehicle	27
1.6 Lookup/Add Customer	28
1.7 Add Vehicle.....	31
1.8 Lookup/Add Vendor	33
1.9 Add Part to Order	34
1.10 Generate Seller History Report	36
1.11 Generate Avg Time in Inventory Report.....	37
1.12 Generate Price Per Condition Report	39
1.13 Generate Parts Statistics Report.....	40
1.14 Generate Monthly Sales/Drilldown Report.....	41

1 Abstract Code + SQL

1.1 Search Vehicles

Abstract Code + SQL

- ❖ The **Vehicle Search/Result Form** will be the front page of the application. It has public access for regular users to perform the search for a car based on certain parameters.
- ❖ Privileged users will be able to log in to access restricted information that is not shown to regular users by authenticated with a valid username and password and clicking on the Login button.
- ❖ Run the **Search Vehicles** task and subtasks using the HTTP Session/Cookie associated with the user as follows:
 - Find all Vehicle records that have not been sold via the SALE ternary relationship (i.e., no SALE relationship instance exists linking that Vehicle to a Customer and Sales Agent), and have no pending parts via the FOR_VEHICLE relationship towards **PartsOrder** and the EXISTS_IN relationship towards **PartsOrderLine**.
 - For regular and privileged users calculate the number of Vehicles available for sale by counting all Vehicle.vin after excluding the number of Vehicles that have Sale.sale_date, and all unique **PartsOrder**.parts_order_number (parts_order_number = vin) where there is still at least one **PartsOrderLine**.status in “ordered” or “received” status (or different to installed).

```
SELECT COUNT(*) AS vehicles_available_for_sale
FROM Vehicle v
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND v.vin NOT IN (
    SELECT DISTINCT po.vin FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
    pol.parts_order_number
    WHERE pol.status != 'Installed'
)
```

- For privileged users acquisition specialist, operating managers, and the owner, the number of Vehicles with missing parts will be calculated as the count of unique **PartsOrder**.parts_order_number (vin) where there is still at least one **PartsOrderLine**.status in “ordered” or “received” status (or different to installed).

The count of Vehicles with pending parts is produced by the following SQL query:

```
SELECT COUNT(DISTINCT po.parts_order_number)
FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
```

```
WHERE pol.status != 'Installed';
```

- The number of cars for sale and the number of cars with missing parts will be shown on the Search Vehicles page depending on the user that accesses it.
- Find all Vehicle records associated with a [VehicleType](#) of car via the IS_OF_TYPE relationship, or a Manufacturer via the MADE_BY relationship, or Vehicle attributes (year, fuel_type, drive_train, or Color via the HAS relationship). For filtering purposes, sale data will be obtained via the SALE ternary relationship (linking [Vehicle](#), [Customer](#), and [Sales Agent](#)), as well as Vehicle records that have pending parts via the FOR_VEHICLE relationship towards [PartsOrder](#) and the EXISTS_IN relationship towards PartsOrderLine.
 - Any users (public/privileged) can search for the following common Vehicle attributes: [Vehicle.Manufacturer.manufacturer_name](#), [Vehicle.VehicleType.type_name](#), [Vehicle.year](#), [Vehicle.fuel_type](#), [Vehicle.drive_train](#), [Vehicle.Color](#). Only one parameter at a time can be used for the search. All these attributes will be shown on the Vehicle Search page when the user lands on it. To ease the search, all these attributes will provide a drilldown functionality only showing unique values found among the recorded attributes.

The unique dropdown values are produced by the following SQL queries (one per attribute):

```
-- Manufacturer dropdown
SELECT DISTINCT m.manufacturer_name
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name = m.manufacturer_name
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND v.vin NOT IN (
SELECT DISTINCT po.vin FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
WHERE pol.status != 'Installed'
)
ORDER BY m.manufacturer_name;

-- Vehicle Type dropdown
SELECT DISTINCT v.vehicle_type_name
FROM Vehicle v
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND v.vin NOT IN (
```

```

SELECT DISTINCT po.vin FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
WHERE pol.status != 'Installed'
)
ORDER BY v.type_name;

-- Year dropdown
SELECT DISTINCT v.year
FROM Vehicle v
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND v.vin NOT IN (
SELECT DISTINCT po.vin FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
WHERE pol.status != 'Installed'
)
ORDER BY v.year;

-- Fuel Type dropdown
SELECT DISTINCT v.fuel_type
FROM Vehicle v
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND v.vin NOT IN (
SELECT DISTINCT po.vin FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
WHERE pol.status != 'Installed'
)
ORDER BY v.fuel_type;

-- Drivetrain dropdown
SELECT DISTINCT v.drive_train
FROM Vehicle v
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND v.vin NOT IN (
SELECT DISTINCT po.vin FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number

```

```

WHERE pol.status != 'Installed'
)
ORDER BY v.drive_train;

-- Color dropdown
SELECT DISTINCT c.color_name
FROM Vehicle v
JOIN VehicleColor vc ON v.vin = vc.vin
JOIN Color c ON vc.color_name = c.color_name
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND v.vin NOT IN (
SELECT DISTINCT po.vin FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
WHERE pol.status != 'Installed'
)
ORDER BY c.color_name;

```

- When the search criteria are fixed, the user will click on **Search** button.
- This search will only show *Vehicles on sale*: all [Vehicle.vin](#) excluding all [PartsOrder.parts_order_number](#) (parts_order_number = vin) where there is still at least one [PartsOrderLine.status](#) in “ordered” or “received” status (or different to installed) and all [Vehicle.vin](#) that have associated a [Sale.sale_date](#).
- The output of a search will be a list of Vehicles with no parts pending and the following attributes: [Vehicle.Manufacturer.manufacturer_name](#) (via MADE_BY relationship), [Vehicle.Type.type_name](#) (via IS_OF_TYPE relationship), [Vehicle.vin](#), [Vehicle.model](#), [Vehicle.year](#), [Vehicle.fuel_type](#), [Vehicle.drive_train](#), [Vehicle.Color.color_name](#) (via HAS relationship), [Vehicle.horse_power](#), and the derived sale_price (calculated as $1.25 \times \text{Purchase.purchase_price} + 1.10 \times \text{total parts cost from PartsOrderLine}$). When a Vehicle HAS several Colors, only one row with all colors listed will be returned. These results are sorted by vin in ascending order. By the side of each car, the user can click on Vehicle Details and the Vehicle Details Form (detail page) will pop up.

The search results described above are produced by the following single SQL query. Since only one attribute filter can be applied at a time, the query uses a @filterType parameter to indicate which attribute the user selected ('manufacturer', 'type', 'year', 'fuel_type', 'drivetrain', or 'color') and a @searchValue parameter to hold the chosen value. Each conditional AND clause acts as a pass-through unless it matches the active filter, ensuring only the selected attribute is applied:

```
SELECT m.manufacturer_name, v.type_name, v.vin, v.model, v.year,
v.fuel_type, v.drive_train,
GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name SEPARATOR
', ') AS colors,
v.horse_power,
(1.25 * p.purchase_price + 1.10 * COALESCE(pc.total_parts_cost, 0))
AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name = m.manufacturer_name
LEFT JOIN VehicleColor hc ON v.vin = hc.vin
LEFT JOIN Color c ON hc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    GROUP BY po.vin
) pc ON pc.vin = v.vin
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s) AND v.vin NOT IN (
    SELECT DISTINCT po.vin
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    WHERE pol.status != 'Installed'
)
AND (@filterType != 'manufacturer' OR m.manufacturer_name =
@searchValue)
AND (@filterType != 'type' OR v.type_name = @searchValue)
AND (@filterType != 'year' OR v.year = @searchValue)
AND (@filterType != 'fuel_type' OR v.fuel_type = @searchValue)
AND (@filterType != 'drive_train' OR v.drive_train = @searchValue)
```



```

AND (@filterType != 'color' OR v.vin IN (
    SELECT hc2.vin FROM VehicleColor hc2 WHERE hc2.color_name =
@searchValue
))
GROUP BY v.vin, m.manufacturer_name, v.type_name, v.model, v.year,
v.fuel_type, v.drive_train, v.horse_power, p.purchase_price,
pc.total_parts_cost
ORDER BY v.vin ASC;

```

- When no Vehicles match the searching criteria, the error message received is:
“Sorry, it looks like we don’t have that in stock will be shown!”
- For the privileged search we will also need to find all **Vehicle** records associated with a **VehicleType** of car via the IS_OF_TYPE relationship, or a Manufacturer via the MADE_BY relationship, or Vehicle attributes (year, fuel_type, drive_train, or Color via the HAS relationship). For filtering purposes, sale data will be obtained via the SALE ternary relationship (linking Vehicle, Customer, and Sales Agent), as well as Vehicle records that have pending parts via the FOR_VEHICLE relationship towards PartsOrder and the EXISTS_IN relationship towards PartsOrderLine.
 - All privileged users can perform a search based on common Vehicle attributes, as regular users can (**Vehicle.Manufacturer**.manufacturer_name, **Vehicle.VehicleType**.type_name, **Vehicle**.year, **Vehicle**.fuel_type, **Vehicle**.drive_train, **Vehicle**.Color):
 - Sales agents and public users : gets only *Vehicles on sale*.
 - Acquisitions specialists: get Vehicles on sale and Vehicles with pending parts (all **Vehicle**.vin except those that have associated a **Sale**.sale_date).

The search results are produced by the following SQL query:

```

SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
    GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ') AS colors,
    v.horse_power,
    (1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin

```

```

LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po JOIN PartsOrderLine pol ON
po.parts_order_number = pol.parts_order_number
    GROUP BY po.vin
) pc ON pc.vin = v.vin WHERE v.vin NOT IN (SELECT s.vin FROM
Sale s) AND m.manufacturer_name = @searchValue
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;

```

- Owner: full access.

The search results are produced by the following SQL query:

```

SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
GROUP_CONCAT(DISTINCT c.color_name
ORDER BY c.color_name SEPARATOR ', ') AS colors,
v.horse_power,
(1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    GROUP BY po.vin
) pc ON pc.vin = v.vin
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND m.manufacturer_name = @searchValue

```

```
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;
```

- Moreover, all privileged users can perform a search in the database matching a particular vin. A designated field for this purpose will be shown in the Search page. When the privileged user introduces the vin (case sensitive), all **Vehicle.vin** will be searched for a match.

The vin search is produced by the following SQL query:

```
SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ') AS colors,
v.horse_power,
(1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
GROUP BY po.vin
) pc ON pc.vin = v.vin
WHERE LOWER(v.vin) = LOWER(@searchvin)
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;
```

- Operating managers: can filter by sold Vehicles, unsold Vehicles, or all Vehicles. Unsold Vehicles include those with pending parts:
 - Sold Vehicles: all `Vehicle.vin` that have associated a valid `Sale.sale_date`.

Produced by the following SQL query:

```
SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ') AS colors,
v.horse_power,
(1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
  SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
  FROM PartsOrder po
  JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
  GROUP BY po.vin
) pc ON pc.vin = v.vin
WHERE v.vin IN (SELECT s.vin FROM Sale s) AND
m.manufacturer_name = @searchValue
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;
```

- Unsold Vehicles: all `Vehicle.vin` that do not have an associated SALE ternary relationship instance (i.e., no `sale_date` exists linking that Vehicle to a Customer and Sales Agent).

Produced by the following SQL query:

```

SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
      GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ') AS colors,
      v.horse_power,
      (1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
      SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
      FROM PartsOrder po JOIN PartsOrderLine pol ON
po.parts_order_number = pol.parts_order_number
      GROUP BY po.vin ) pc ON pc.vin = v.vin WHERE v.vin NOT
IN (SELECT s.vin FROM Sale s) AND m.manufacturer_name =
@searchValue
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;

```

- All Vehicles: all `Vehicle.vin` in the database.

Produced by the following SQL query:

```

SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
      GROUP_CONCAT(DISTINCT c.color_nameORDER BY c.color_name
SEPARATOR ', ') AS colors,
      v.horse_power,
      (1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name

```

```

LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number GROUP BY po.vin
) pc ON pc.vin = v.vin
WHERE m.manufacturer_name = @searchValue
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;

```

- When the search criteria are fixed, the user will click on **Search** button.
 - In all cases, the search request will provide
Vehicle.Manufacturer.manufacturer_name, Vehicle.VehicleType.type_name,
Vehicle.vin, Vehicle.model, Vehicle.year, Vehicle.Fuel_type, Vehicle.Drive_train,
Vehicle.Color, Vehicle.horse_power, and the derived sale_price (calculated as
 $1.25 \times \text{Purchase.purchase_price} + 1.10 \times \text{total parts cost from PartsOrderLine}$).
When Vehicle.Color stores several colors, only one row with all colors listed will
be returned. These results are sorted by vin in ascending order. By the side of
each car, the user can click on Vehicle Details and the Vehicle Details Form
(detail page) will pop up.
 - When no Vehicles match the searching criteria, the error message received is:
“Sorry, it looks like we don’t have that in stock will be shown!”
- Find all Vehicle records that match a keyword entered by the user across Manufacturer
via the MADE_BY relationship, or Vehicle attributes (Year, Model, or Notes). For filtering
purposes, sale data will be obtained via the SALE ternary relationship (linking Vehicle,
Customer, and Sales Agent), as well as Vehicle records that have pending parts via the
FOR_VEHICLE relationship towards PartsOrder and the EXISTS_IN relationship towards
PartsOrderLine.
- The user will provide a keyword (case sensitive) in the designated field for that
purpose in the Search Page. When the search criteria are fixed, the user will click
on **Search** button.

- The keyword is then compared with all words stored in `Vehicle.Manufacturer.manufacturer_name`, `Vehicle.model`, `Vehicle.year`, and `Vehicle.notes` parameters that the user could access to:
 - Public users and sales agents will just get the *Vehicles on sale*.

The keyword search results for public users and sales agents are produced by the following SQL query:

```
SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ') AS colors,
v.horse_power,
(1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    GROUP BY po.vin
) pc ON pc.vin = v.vin
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND v.vin NOT IN (
    SELECT DISTINCT po.vin
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    WHERE pol.status != 'Installed'
)
AND ( LOWER(m.manufacturer_name) LIKE
LOWER(CONCAT('%', @keyword, '%'))
```

```

        OR LOWER(v.model) LIKE LOWER(CONCAT('%', @keyword,
'%'))
        OR CAST(v.year AS CHAR) LIKE CONCAT('%', @keyword,
'%')
        OR LOWER(v.notes) LIKE LOWER(CONCAT('%', @keyword,
'%'))
    )
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;

```

- Acquisition managers: Vehicles on sale and Vehicles with missing parts (unsold Vehicles).

The keyword search results for acquisition specialists are produced by the following SQL query, which removes the pending-parts exclusion so that unsold Vehicles with parts pending are also included:

```

SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
    GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ') AS colors,
    v.horse_power,
    (1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    GROUP BY po.vin )
pc ON pc.vin = v.vin

```



```

WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND (
    LOWER(m.manufacturer_name) LIKE LOWER(CONCAT('%',
@keyword, '%'))
    OR LOWER(v.model) LIKE LOWER(CONCAT('%', @keyword,
'%'))
    OR CAST(v.year AS CHAR) LIKE CONCAT('%', @keyword,
'%')
    OR LOWER(v.notes) LIKE LOWER(CONCAT('%', @keyword,
'%'))
)
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;

```

- Operating managers: sold, unsold and all Vehicles.

The keyword search results for operating managers are produced by the following three SQL queries depending on the filter selected:

Sold Vehicles:

```

SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
    GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ') AS colors,
    v.horse_power,
    (1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po

```

```

        JOIN PartsOrderLine pol ON po.parts_order_number =
        pol.parts_order_number
        GROUP BY po.vin
    ) pc ON pc.vin = v.vin
WHERE v.vin IN (SELECT s.vin FROM Sale s)
AND (
    LOWER(m.manufacturer_name) LIKE LOWER(CONCAT('%',
@keyword, '%'))
    OR LOWER(v.model) LIKE LOWER(CONCAT('%', @keyword, '%'))
    OR CAST(v.year AS CHAR) LIKE CONCAT('%', @keyword, '%')
    OR LOWER(v.notes) LIKE LOWER(CONCAT('%', @keyword, '%'))
)
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;

```

Unsold Vehicles:

```

SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
    GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ' ) AS colors,
    v.horse_power,
    (1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0))
AS sale_price FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    GROUP BY po.vin

```

```

) pc ON pc.vin = v.vin
WHERE v.vin NOT IN (SELECT s.vin FROM Sale s)
AND (
    LOWER(m.manufacturer_name) LIKE LOWER(CONCAT('%',
@keyword, '%'))
    OR LOWER(v.model) LIKE LOWER(CONCAT('%', @keyword,
'%'))
    OR CAST(v.year AS CHAR) LIKE CONCAT('%', @keyword,
'%')
    OR LOWER(v.notes) LIKE LOWER(CONCAT('%', @keyword,
'%'))
)
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;

```

All Vehicles :

```

SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
    GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ') AS colors,
    v.horse_power,
    (1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number

```

```

        GROUP BY po.vin
    ) pc ON pc.vin = v.vin
WHERE (
    LOWER(m.manufacturer_name) LIKE LOWER(CONCAT('%',
@keyword, '%'))
    OR LOWER(v.model) LIKE LOWER(CONCAT('%', @keyword, '%'))
    OR CAST(v.year AS CHAR) LIKE CONCAT('%', @keyword, '%')
    OR LOWER(v.notes) LIKE LOWER(CONCAT('%', @keyword, '%'))
) GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;

```

- Owner: full access.

Produced by the following SQL query:

```

SELECT m.manufacturer_name, v.vehicle_type_name, v.vin,
v.model, v.year, v.fuel_type, v.drive_train,
    GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name
SEPARATOR ', ' ) AS colors,
    v.horse_power,
    (1.25 * p.purchase_price + 1.10 *
COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
JOIN Manufacturer m ON v.manufacturer_name =
m.manufacturer_name
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    GROUP BY po.vin
) pc ON pc.vin = v.vin
WHERE m.manufacturer_name = @searchValue

```

```
GROUP BY v.vin, m.manufacturer_name, v.vehicle_type_name,
v.model, v.year, v.fuel_type, v.drive_train, v.horse_power,
p.purchase_price, pc.total_parts_cost
ORDER BY v.vin ASC;
```

- In all cases, the search request will provide [Vehicle.Manufacturer.manufacturer_name](#), [Vehicle.VehicleType.type_name](#), [Vehicle.vin](#), [Vehicle.model](#), [Vehicle.year](#), [Vehicle.fuel_type](#), [Vehicle.drive_train](#), [Vehicle.Color](#), [Vehicle.horse_power](#), and derived sale_price (calculated as $1.25 \times \text{Purchase.purchase_price} + 1.10 \times \text{total parts cost from PartsOrderLine}$) for the Vehicles that have a match in any of the attributes with the keyword provided by the user. When [Vehicle.Color](#) stores several colors, only one row with all colors listed will be returned. These results are sorted by vin in ascending order. By the side of each car, the user can click on **Vehicle Details** and the **Vehicle Details Form** (detail page) will pop up.
- When no Vehicles match the searching criteria, the error message received is: “Sorry, it looks like we don’t have that in stock will be shown!”

1.2 Authenticate User

Abstract Code + SQL

- ❖ Privileged users can authenticate to access specific features associated with each role by filling username and password in the designated fields shown in the **Vehicle Search/Result Form** and then pressing the Login button.
- ❖ Run the **Authenticate User** task as follows for each login attempt:
 - Obtain username ('\$Username') and password ('\$Password') introduced by the user when the Login button is pressed.
 - If [StaffUser.username](#) = '\$Username' and [StaffUser.password_plaintext](#) = '\$Password' then allow access for the privileged user:
 - Record session information and user role ([AcquisitionSpecialist/SaleS_AGENT/OperatingManager](#)) and refresh **Vehicle Search/Result Form** allowing access to appropriate functionalities.

The authentication and role retrieval is produced by the following SQL query. It matches the username and password against [StaffUser](#), and LEFT JOINS each role subtype table to determine which role(s) the user holds. If the query returns one row, authentication succeeds and the role flags are stored in the session:

```
SELECT su.username, su.first_name, su.last_name,
CASE WHEN acq.username IS NOT NULL THEN 1 ELSE 0 END AS
is_acquisition_specialist,
```

```

CASE WHEN sa.username IS NOT NULL THEN 1 ELSE 0 END AS
is_sales_agent,
CASE WHEN om.username IS NOT NULL THEN 1 ELSE 0 END AS
is_operating_manager
FROM StaffUser su
LEFT JOIN AcquisitionSpecialist acq ON su.username =
acq.username
LEFT JOIN SalesAgent sa ON su.username = sa.username
LEFT JOIN OperatingManager om ON su.username = om.username
WHERE su.username = @Username AND su.password_plaintext =
@Password;

```

- If `StaffUser.username != '$Username'` or `StaffUser.username = '$Username'` and `StaffUser.password_plaintext != '$Password'`:
 - Refresh **Vehicle Search/Result Form** and provide generic login error message: "Incorrect username or password".

1.3 View Vehicle Details

Abstract Code + SQL

- ❖ User navigated to the **Vehicle Details** form associated with the **View Vehicle Details** task by selecting a search result on the **Vehicle Search** form, and clicking button **Vehicle Details**. The '\$vin' gets passed in.
- ❖ The **Mother** task:
 - Calculates the `sale_price` as follows:
 - Find the current **Vehicle** using `Vehicle.vin`
 - Find the **PartsOrder** entities associated with the current **Vehicle**.
 - For each **PartsOrder**:
 - Find all **PartsOrderLines**.
 - Calculate the `total_parts_cost` using the `unit_price` and `quantity` attributes of each **PartsOrderLine**.
 - Find the **Purchase** associated with the **Vehicle** to get the `purchase_price`.
 - Calculate the `sale_price`. The formula is:
 - $\text{sale_price} = 1.25 \times \text{purchase_price} + 1.10 * \text{total_parts_cost}$
 - Runs the **Get Basic Vehicle Information** task, passing in the '\$vin' and '\$sale_price':
 - Query for information about the **Vehicle**:
 - Display the **Vehicle** single-value attributes of `vin`, `Model`, `Year`, `Fuel_type`, `Drivetrain`, `Horsepower`, `Notes`
 - Find the **COLOR(s)** of the **Vehicle** and display all `color_name` attributes in a single result row list.
 - Find the **VehicleType** of the **Vehicle** and display the `type_name` attribute.
 - Find the **Manufacturer** of the **Vehicle** and display `manufacturer_name` attribute.

- Display Sale Price.
- The query for the above steps is as follows:

```
SELECT
    v.vin, v.model, v.`year`, v.`condition`, v.fuel_type,
    v.drive_train, v.horse_power, v.notes,
    v.manufacturer, v.vehicle_type_name,
    GROUP_CONCAT(DISTINCT c.color_name ORDER BY c.color_name SEPARATOR ',
    ') AS colors, (1.25 * p.purchase_price + 1.10 *
    COALESCE(pc.total_parts_cost, 0)) AS sale_price
FROM Vehicle v
LEFT JOIN VehicleColor vc ON v.vin = vc.vin
LEFT JOIN Color c ON vc.color_name = c.color_name
LEFT JOIN Purchase p
ON p.vin = v.vin
LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
    pol.parts_order_number
    GROUP BY po.vin
) pc ON pc.vin = v.vin
WHERE v.vin = '$vin'
GROUP BY
    v.vin, v.model, v.`year`, v.`condition`, v.fuel_type,
    v.drive_train, v.horse_power, v.notes,
    v.manufacturer, v.vehicle_type_name, p.purchase_price;
```

- If the user is not logged in, we are done and can wait for a click on **Back to Search** button.
- ❖ If the user is logged in as a **SalesAgent**:
 - If the **Vehicle** is unsold and all **PartsOrderLine** records have status = 'installed', display the **Sell Vehicle** button.
 - Check whether the vehicle is sold:

```
SELECT vin FROM Sale WHERE vin = '$vin';
```

- Check whether the vehicle has pending parts:

```
SELECT COUNT(*) AS pending_parts
```

```
FROM PartsOrderLine pol INNER JOIN PartsOrder po ON
po.parts_order_number = pol.parts_order_number
WHERE po.vin = '$vin' AND pol.`status` != 'Installed';
```

- On click **Sell Vehicle button**, go to the **Lookup/Add Customer** form, passing '\$vin'.
- ❖ If the user is logged in as an **AcquisitionSpecialist**, the mother task runs the **Get Acquisition Specialist Information task**, passing the **Vehicle** and information on **PartsOrder(s)**, **PartsOrderLine(s)** and **total_parts_cost**:
 - Find the **Purchase** associated with the current **Vehicle**, and display **purchase_price** and **purchase_date**.

```
SELECT
    p.purchase_price, p.purchase_date, p.seller_identifider,
    p.handler_username, su.first_name AS specialist_first_name,
    su.last_name AS specialist_last_name
FROM Purchase p
JOIN StaffUser su ON su.username = p.handler_username
WHERE p.vin = '$vin';
```

- For each **PartsOrderLine**:
 - Display the **vendor_part_number**, **description**, **unit_price**, **quantity**, and **status** attributes, as well as the **PartOrder** it came from.
 - Find the **Vendor** associated with each **PartsOrder** and display **vendor_name**.
 - If status is not 'installed', display the **Update Status** button for that **PartsOrderLine** row.

```
SELECT
    po.parts_order_number, po.order_ordinal, po.vendor_name,
    pol.vendor_part_number, pol.description, pol.unit_price,
    pol.quantity, pol.status, (pol.unit_price * pol.quantity) AS
line_total
FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
WHERE po.vin = '$vin'
ORDER BY po.order_ordinal, pol.vendor_part_number;
```

- Display **Add Parts Order** button.

```
SELECT
    COALESCE(SUM(pol.unit_price * pol.quantity), 0) AS
total_parts_cost
```



```

FROM PartsOrder po
JOIN PartsOrderLine pol
ON po.parts_order_number = pol.parts_order_number
WHERE po.vin = '$vin';

```

- On click **Update Status button**: Go to the **Modify Part Status** form, passing parts_order_number and vendor_part_number.
- On click **Add Parts Order** button: Go to the **Add Parts Order** form, passing '\$vin'.
- ❖ If the user is logged in as an **OperatingManager**, run the **Get Operating Manager Information** task, passing in '\$vin':
 - Find the **Customer** from whom the **Vehicle** was bought.
 - If **Customer** is an **IndividualCustomer**, display first_name and last_name.
 - If **Customer** is a **BusinessCustomer**, display business_name.
 - Display **Customer** street, city, state, postal_code, phone, email. Do not display ssn or tax_id.

```

-- Get total parts cost
SELECT c.phone_number, c.email, c.street, c.city, c.state,
c.postal_code,
      COALESCE(
        CONCAT(ic.first_name, ' ', ic.last_name), bc.business_name
      ) AS seller_name
FROM Purchase p
JOIN Customer c
ON c.customer_id = p.seller_identifiser
LEFT JOIN IndividualCustomer ic
ON ic.customer_id = c.customer_id
LEFT JOIN BusinessCustomer bc
ON bc.customer_id = c.customer_id
WHERE p.vin = '$vin';

```

- If a **Sale** record exists for this **Vehicle**:
 - Display **Sale**.sale_date and the derived sale_price.
 - Find the **SalesAgent** who executed the **Sale** and display first_name and last_name.
 - Find the **Customer** to whom the **Vehicle** was sold.
 - If the **Customer** is an **IndividualCustomer**, display first_name and last_name.
 - If the **Customer** is a **BusinessCustomer**, display business_name.
 - Display buyer street, city, state, postal_code, phone, email.
 - Do not display ssn or tax_id.

```

SELECT
    s.sale_date,
    su.first_name AS agent_first_name,
    su.last_name AS agent_last_name,
    c.phone_number AS buyer_phone,
    c.email AS buyer_email,
    c.street AS buyer_street,
    c.city AS buyer_city,
    c.state AS buyer_state,
    c.postal_code AS buyer_postal_code,
    COALESCE(
        CONCAT(ic.first_name, ' ', ic.last_name),
        bc.business_name
    ) AS buyer_name
FROM Sale s
JOIN StaffUser su
ON su.username = s.sales_agent_username
JOIN Customer c
ON c.customer_id = s.buyer_id
LEFT JOIN IndividualCustomer ic
ON ic.customer_id = c.customer_id
LEFT JOIN BusinessCustomer bc
ON bc.customer_id = c.customer_id
WHERE s.vin = '$vin';

```

- Find and display the [AcquisitionSpecialist](#) associated with the [Vehicle](#).
- No action buttons are displayed. When ready, [OperatingManager](#) user clicks the Back to Search button to return to the [Vehicle Search](#) form.
- ❖ If the user is logged in as an [Owner](#):
 - Run the **Get Acquisition Specialist Information** task passing in [Vehicle](#) and information on [PartsOrder](#)(s) [PartsOrderLine](#)(s) and total_parts_cost,
 - Run the **Get Operating Manager Information** task passing in '\$vin'.
 - If the [Vehicle](#) is unsold and all [PartsOrderLine](#) records have status = 'installed', display the **Sell Vehicle** button.
 - As per tasks above:
 - On click **Sell Vehicle button**, go to the [Lookup/Add Customer](#) form, passing [Vehicle](#).vin.
 - On click **Update Status button**: Go to the [Modify Part Status](#) form, passing parts_order_number and vendor_part_number.
 - On click **Add Parts Order** button: Go to the [Add Parts Order](#) form, passing '\$vin'.
- ❖ Finally for all users:

- On click **Back to Search** button: Return to the Vehicle Search form (all user categories).

1.4 Modify Part Status

Abstract Code + SQL

- ❖ User navigated to the Modify Part Status form associated with the Modify Part Status task by clicking the **Update Status** button on the Vehicle Details form. The parts_order_number and vendor_part_number attributes are passed in from the **View Vehicle Details** mother task.
- ❖ Run the **Modify Part Status** task as follows:
 - View Vehicle Details will display PartsOrderLine associated with the Vehicle: parts_order_number, vendor_name, vendor_part_number, description, unit_price, quantity, status.
 - Find the current PartsOrderLine that needs modification based on parts_order_number and vendor_part_number based on customer input.
 - Display the *New Status* input field.
 - On click **Update Status** button:
 - Validate that new_status represents a valid forward transition from the current status. If not, display an error message and remain on the Modify Part Status form.
 - If valid, update PartsOrderLine.status to new_status.
 - Return to the Vehicle Details form for current Vehicle.
 - On click **Cancel** button:
 - Return to the Vehicle Details form for current Vehicle without making any changes.

```
--dropdowns
SELECT `status` FROM PartsOrderLine ORDER BY `status`;

--assume that $parts_order_number and $vendor_part_number of current
vehicle in vehicle details page is managed by the application.
SELECT vendor_part_number, `description`, `status`, quantity, unit_price,
parts_order_number
FROM PartsOrderLine
WHERE vendor_part_number = '$vendor_part_number' AND parts_order_number =
'$parts_order_number';

--update with new status (dropdown)
if $new_status is correct
    UPDATE PartsOrderLine
    SET status = '$new_status'
```

```

WHERE vendor_part_number = '$vendor_part_number' AND
'parts_order_number' = '$parts_order_number';
else
    Error message

```

1.5 Confirm Sale of Vehicle

Abstract Code + SQL

- ❖ User navigated to the **Confirm Sale of Vehicle** form associated with the **Confirm Sale of Vehicle** task by clicking the **Sell Vehicle** button on the **Vehicle Details** form, proceeding through the **Lookup/Add Customer** task, and clicking the **Confirm** button on the **Lookup/Add Customer** form. **Vehicle.vin** and **Sale_price** as well as **Customer_id** are passed in from the **Lookup/Add Customer** task.
- ❖ Run the **Confirm Sale of Vehicle** task as follows:
 - Find the current **Vehicle** using **vin**.
 - Verify that the Vehicle is unsold (no **Sale** record exists for this **Vehicle** via **SOLD_VEHICLE**).

```

SELECT COUNT(*) AS sale_count
FROM Sale
WHERE vin = '$vin';

```

- Verify that all **PartsOrderLine** records associated with this **Vehicle** have status = 'Installed'.

```

SELECT COUNT(*) AS pending_count
FROM PartsOrder po
JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
WHERE po.vin = '$vin' AND pol.status != 'Installed';

```

- If either condition is not met, display an error message and return to the **Vehicle Details** form.
- Find the **Customer** identified by the **Lookup/Add Customer** task, using the **customer_id** passed in from that task.
- Display **vin** and the **Sale_price**.

```

SELECT v.vin, p.purchase_date, p.purchase_price,
(1.25 * p.purchase_price + 1.10 *
COALESCE(parts.total_parts_cost, 0)) AS sale_price

```

```

FROM Vehicle v
JOIN Purchase p ON v.vin = p.vin
LEFT JOIN (
    SELECT po.vin, SUM(pol.unit_price * pol.quantity) AS
total_parts_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    GROUP BY po.vin
) parts ON v.vin = parts.vin
WHERE v.vin = '$vin';

```

- If the Customer is an **IndividualCustomer**, display first_name and last_name.
- If the Customer is a **BusinessCustomer**, display business_name.

```

SELECT
    COALESCE(
        CONCAT(ic.first_name, ' ', ic.last_name),
        bc.business_name
    ) AS customer_name
FROM Customer c
LEFT JOIN IndividualCustomer ic ON c.customer_id = ic.customer_id
LEFT JOIN BusinessCustomer bc ON c.customer_id = bc.customer_id
WHERE c.customer_id = '$customer_id';

```

- Display the *sale_date* input field.
- On click **Confirm Sale** button: Validate that sale_date is a valid date. If not, display an error message and remain on the **Confirm Sale of Vehicle** form. If valid, insert a new **Sale** record with sale_date, vin, buyer_id, and the current staff username via EXECUTED_BY.

```

INSERT INTO Sale (vin, sale_date, buyer_id, sales_agent_username)
VALUES ('$vin', '$sale_date', '$customer_id', '$username');

```

- Return to the **Vehicle Details** form for the current **Vehicle**.
- On click Cancel button: Return to the **Vehicle Details** form for the current **Vehicle** without making any changes.

1.6 Lookup/Add Customer

Abstract Code + SQL

- ❖ The **Lookup/Add Customer** task handles customer selection both when buying and selling a **Vehicle**. An **AcquisitionSpecialist** or **OWNER** user interacts with this task via the **Add Vehicle** form. A **SalesAgent** or **OWNER** user interacts with this task via the **Confirm Sale** form. Both contexts present an interface allowing the user to search for a **Customer** by SSN or Tax_ID.
- ❖ Run the **Lookup/Add Customer** task as follows, passing in context (**Add Vehicle** or **Confirm Sale**):
 - Display search interface on the relevant form with radio buttons for search type (SSN or Tax_ID) and *SSN/Tax_id* input search field.
 - User selects search type (SSN or Tax_ID radio button) and enters value in search field.
 - On click Search button:
 - Check that radio button is selected and validate search field. If validation fails, show error message and remain on original form with search interface.
 - If searching by SSN, query **IndividualCustomer** by SSN.

```

SELECT
    c.customer_id,
    ic.first_name,
    ic.last_name,
    ic.ssn,
    c.email,
    c.phone_number,
    c.street,
    c.city,
    c.state,
    c.postal_code
FROM Customer c
JOIN IndividualCustomer ic ON c.customer_id = ic.customer_id
WHERE ic.ssn = '$ssn';

```

- If searching by Tax_ID, query **BusinessCustomer** by tax_id.

```

SELECT
    c.customer_id,
    bc.business_name,
    bc.primary_contact_first_name,
    bc.primary_contact_last_name,
    bc.primary_contact_title,
    bc.tax_id,
    c.email,
    c.phone_number,
    c.street,
    c.city,

```

```

        c.state,
        c.postal_code
FROM Customer c
JOIN BusinessCustomer bc ON c.customer_id = bc.customer_id
WHERE bc.tax_id = '$tax_id';

```

- If matching record found, display Customer details:
 - For **IndividualCustomer**: Display First_name, Last_name, Street, City, State, Postal_code, phone_number, and Email (optional).
 - For **BusinessCustomer**: Display business_name, Primary_contact_first_name, Primary_contact_last_name, Primary_contact_title, Street, City, State, Postal_code, phone_number, Email (optional).
- On click:
 - **Confirm** button: Pass customer subtype (**IndividualCustomer** or **BusinessCustomer**), and **Customer** key (SSN or Tax_id value) to the calling task (Add Vehicle or Confirm Sale of Vehicle, depending on context).
 - **Search Again** button: Return to original form with search interface.
- If no record found, display **Add Customer** form with radio button to select **IndividualCustomer** or **BusinessCustomer**. Radio button defaults to the search type originally selected. Display input fields:
 - Common fields for both types: Street, City, State, Postal_code, phone_number, Email (optional).
 - If **IndividualCustomer** selected: First_name, Last_name, SSN.
 - If **BusinessCustomer** selected: business_name, Primary_contact_first_name, Primary_contact_last_name, Primary_contact_title, Tax_id.
- On click:
 - **Save** button: Validate all required fields are filled and uniqueness constraint is satisfied (SSN or Tax_id must not already exist in database). If validation fails, display error message and remain on **Add Customer** form. If valid, insert new **Customer** and subtype (**IndividualCustomer** or **BusinessCustomer**) record:

```

INSERT INTO Customer (phone_number, email, state, city,
street, postal_code)
VALUES ('$phone_number', '$email', '$state', '$city',
'$street', '$postal_code');

```

- Retrieve the auto-generated customer_id using LAST_INSERT_ID() for use in the subtype insert.
- If IndividualCustomer:

```
INSERT INTO IndividualCustomer (customer_id, ssn, first_name,
last_name)
VALUES (LAST_INSERT_ID(), '$ssn', '$first_name',
'$last_name');
```

- If BusinessCustomer:

```
INSERT INTO BusinessCustomer (customer_id, tax_id,
business_name, primary_contact_title,
primary_contact_first_name, primary_contact_last_name)
VALUES (LAST_INSERT_ID(), '$tax_id', '$business_name',
'$primary_contact_title', '$primary_contact_first_name',
'$primary_contact_last_name');
```

- Pass **Customer** subtype and key (SSN or Tax_id value) to the calling task (**Add Vehicle** or **Confirm Sale**, depending on context).
 - **Cancel** button: Return to original context form with search interface.
- On click Cancel button:
- Clear inputs and remain on **Add Vehicle** or **Confirm Sale** form

1.7 Add Vehicle

Abstract Code + SQL

- ❖ User navigated to **Add Vehicle** form associated with the **Add Vehicle** task by clicking on the **Add Vehicle** button on the **Vehicle Search** form and completing the **Lookup/Add Customer** task. The **Customer** subtype and key (SSN or Tax_id), and context (Add Vehicle) get passed in from the **Lookup/Add Customer** task.
- ❖ Run the **Add Vehicle** task as follows:
 - Display **Add Vehicle** form with confirmed seller name, input fields for Vehicle details, and Purchase_date.
 - Display the following dropdown fields populated from database:
 - *Type* showing all **VehicleType** options.
 - *Manufacturer* dropdown showing all **Manufacturer** options.
 - *Color* with all COLOR options, with **Add Color** button for multiple colors.

```
SELECT type_name FROM VehicleType ORDER BY type_name ASC;
SELECT manufacturer_name FROM Manufacturer ORDER BY
manufacturer_name ASC;
SELECT color_name FROM Color ORDER BY color_name ASC;
```


- Populate the following dropdown fields based on the enums defined in the database schema:
 - *Condition* with following options: Excellent, Very Good, Good, Fair, Rough.
 - *Fuel Type* with following options: Gas, Diesel, Natural Gas, Hybrid, Plugin Hybrid, Battery, Fuel Cell.
 - *Drivetrain* with options: 4WD, AWD, FWD, RWD.
- User selects values from all of the above dropdown fields.
- User fills in vin, Model_name, Model_year, Horsepower, Notes (optional), Purchase_price, and Purchase_date.
- On click **Submit** button:
 - Verify all required inputs have been provided, and validate all input fields.
 - Verify year is 4 digit integer not greater than current year + 1.
 - Verify horsepower is a positive integer.
 - Verify at least one color is selected.
 - Verify vin is new. If vin already exists (when query below returns a record), display error and remain on **Add Vehicle** form:

```
SELECT vin from Vehicle WHERE vin= '$vin'
```

- If valid:
 - Insert new Vehicle record with values entered by user.

```
INSERT INTO Vehicle (
    vin, model, `year`, `condition`, fuel_type, drive_train,
    horse_power,
    notes, manufacturer_name, vehicle_type_name
)
VALUES (
    '$vin', '$model', '$year', '$condition', '$fuel_type',
    '$drive_train', '$horse_power', '$notes', '$manufacturer',
    '$vehicle_type_name'
);
```

- Insert one row per color selected by user:

```
INSERT INTO VehicleColor (vin, color_name) VALUES ('$vin',
'$color_name');
```

- Insert new **Purchase** record with purchase_price and purchase_date, linking to seller and acquisition specialist:

```
INSERT INTO Purchase (vin, purchase_price,
purchase_date, seller_identifier, handler_username)
VALUES ('$vin', '$purchase_price', '$purchase_date',
'$SellerID', '$SessionUsername');
```

- Go to **Vehicle Details** form for the new **Vehicle**, passing '\$vin'.
- On click **Cancel** button: Return to **Vehicle Search** form.

1.8 Lookup/Add Vendor

Abstract Code + SQL

- ❖ The acquisition specialists, or the owner, successfully authenticate using username and password tuple by clicking the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Vehicle Details Form** accessed through **Vehicle Search/Result Form**. In the **Vehicle Details Form**, the **Add Parts Order** button will launch the **Add Parts Order Form**. The **Add Parts Order Form** will enable the addition/search of Vendors using **Add/Search Vendor** button. Once the vendor is selected/added, it will be linked to the part order.
- ❖ Run the **Lookup/Add Vendor** task and subtasks using the HTTP Session/Cookie associated with the user as follows:
 - The user can search for the vendor's name using **Add Parts Order** button (all **Vendor**.vendor_name should be unique). The vendor's name introduced by the user is then compared to all **Vendor**.vendor_name. If the search is positive, all attributes (vendor_name, vtreet, phone, postal_code, city, and state) for that vendor **Vendor** will be shown. This user could now link this vendor to current **PartsOrder** (parts_order_number + order_ordinal) via the FULFILLS relationship.
 - In the case that the vendor does not exist in the database yet, the user can add it by filling in all required attributes for the new **Vendor**: vendor_name, street, phone, postal_code, city, and state. A consistency check will be performed prior to the addition of new records to **Vendor** by cross checking that the introduced vendor name is not yet an existing **Vendor**.vendor_name. When the vendor is added, it is automatically linked to current **PartsOrder** (parts_order_number + order_ordinal).
- ❖ Once the user finishes searching/adding the vendor, it will be possible to go back to the **Add Parts Order Form** using the **Back to Add Parts** button. Once the user finishes adding parts, they can return to the **Vehicle Details Form** by using the **Back to Vehicle Details** button. If required, the user can go back to **Vehicle Search/Result Form** by clicking on **Back to Search** button.

--dropdowns

```
SELECT DISTINCT vendor_name FROM Vendor ORDER BY vendor_name;
```

--assume that \$vin of current vehicle in vehicle details page is managed by the application and vendor_name is introduced by the user

```

-- search for a vendor
SELECT vendor_name FROM Vendor WHERE vendor_name = '$vendor_name';
START TRANSACTION;
INSERT INTO PartsOrder (parts_order_number, order_ordinal, vin,
vendor_name)
VALUES ('$vin', (SELECT IFNULL(MAX(order_ordinal), 0) + 1 FROM PartsOrder
WHERE vin = '$vin'), '$vin', '$vendor_name');
COMMIT;

-- add a vendor
START TRANSACTION;
INSERT INTO Vendor (vendor_name, phone_number, `state`, city, street,
postal_code)
VALUES ('$vendor_name', '$phone_number', '$state', '$city', '$street',
'$postal_code');
INSERT INTO PartsOrder (parts_order_number, order_ordinal, vin,
vendor_name)
VALUES ('$vin', (SELECT IFNULL(MAX(order_ordinal), 0) + 1 FROM PartsOrder
WHERE vin = '$vin'), '$vin', '$vendor_name');
COMMIT;

```

1.9 Add Part to Order

Abstract Code + SQL

- ❖ The acquisition specialists, or the owner, successfully authenticate using username and password tuple by clicking the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Vehicle Details Form** accessed through **Vehicle Search/Result Form**. In the **Vehicle Details Form**, the **Add Parts Order** button will launch the **Add Parts Order Form**.
- ❖ As explained in the previous section, the **Add Parts Order Form** will enable the addition/search of Vendors using **Add/Search Vendor** button.
- ❖ Run the **Add Parts Order** task and subtasks using the HTTP Session/Cookie associated with the user as follows:
 - When the user wants to add new parts to a Vehicle, a new **PartsOrder** will be generated, and as many **PartsOrder** as the user would like to add to the order via EXISTS_IN relationship. Additionally, **PartsOrder** will be associated with **Vehicle** via the FOR_VEHICLE Relationship.
 - The user will fill in the information about the parts required for a particular order. **PartsOrder.parts_order_number** will be filled in with the vin of the Vehicle and **PartsOrder.order_ordinal** will be automatically added to the part order. **PartsOrder.order_ordinal** is initialized to one for a new vin and is incremented by one unit (+1) every time an order part is completed.

- As part of the process, the vin introduced by the user will be cross checked with existing [Vehicle.vin](#) values. It will only be possible to successfully create a part order if the [Vehicle.vin](#) introduced already exists.
 - If the [Vehicle.vin](#) exists, it will be linked to current [PartsOrder](#) (parts_order_number + order_ordinal) via FOR_VEHICLE relationship.
 - The user will need to keep adding information to the order part to get it completed: [PartsOrderLine.vendor_part_number](#), [PartsOrderLine.description](#), [PartsOrderLine.quantity](#), and [PartsOrderLine.unit_price](#).
 - The [PartsOrderLine.status](#) will be automatically initialized to “ordered” for all newly created parts.
- The subtask **Lookup/Add Vendor** is treated as a separate task and described in its own section. It will be used to link the order part to a [Vendor](#) via the relationship FULFILLS.
- ❖ Once the user finishes adding parts, it will be possible to return to the **Vehicle Details Form** by using the ***Back to Vehicle Details*** button. If required, the user can go back to **Vehicle Search/Result Form** by clicking on ***Back to Search*** button.

```
--dropdowns
SELECT DISTINCT vendor_name FROM Vendor ORDER BY vendor_name;

--assume that $vin of current vehicle in vehicle details page is managed
by the application
SELECT PartsOrder.parts_order_number, PartsOrder.order_ordinal,
Vendor.vendor_name, PartsOrderLine.vendor_part_number,
PartsOrderLine.`description`, PartsOrderLine.`status`,
PartsOrderLine.quantity, PartsOrderLine.unit_price
FROM PartsOrder
INNER JOIN PartsOrderLine ON PartsOrder.parts_order_number =
PartsOrderLine.parts_order_number
INNER JOIN Vendor ON Vendor.vendor_name = PartsOrder.vendor_name
WHERE PartsOrder.vin = '$vin';

--assume that $vin of current vehicle in vehicle details page is managed
by the application and that $vendor_name (dropdown),
$vendor_part_number, $description, $quantity, and $unit_price are
introduced by the user
START TRANSACTION;
INSERT INTO PartsOrder (parts_order_number, order_ordinal, vin,
vendor_name)
VALUES ('$vin', (SELECT IFNULL(MAX(order_ordinal), 0) + 1 FROM
PartsOrder WHERE vin = '$vin'), '$vin', '$vendor_name');
```

```

INSERT INTO PartsOrderLine (vendor_part_number, `description`, `status`,
quantity, unit_price, parts_order_number)
VALUES ('$vendor_part_number', '$description', 'Ordered', '$quantity', '$unit_price', '$parts_order_number');
COMMIT;

```

1.10 Generate Seller History Report

Abstract Code + SQL

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple using the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Seller History Report** form associated with the **Generate Seller History Report** task by first clicking on the **Reports** button on the **Vehicle Search/Results** form, selecting the Seller History option on the **Reports Selection** form, and clicking the **View Report** button.
- ❖ Run the **Generate Seller History Report** task as follows:
 - Find all **Customer** records that have sold at least one **Vehicle** via FROM_SELLER.
 - For each **Customer**:
 - If the **Customer** is an **IndividualCustomer**, display First_name and Last_name as a single Seller_name column.
 - If the **Customer** is a **BusinessCustomer**, display business_name as the Seller_name column.
 - Count the number of **Vehicles** sold by this **Customer** and display as Total_VEHICLES.
 - Calculate the average Purchase_price across all **PURCHASE** records for this Customer's **Vehicles** and display as Avg_purchase_price.
 - Find all **PartsOrder** records associated with this Customer's **Vehicles** via FOR_VEHICLE.
 - For each **PartsOrder**, find all **PartsOrderLine** records via CONTAINS.
 - Calculate the average total quantity of parts (sum of quantity across all **PartsOrderLine** records, divided by Total_VEHICLES) and display as Avg_parts_quantity.
 - Calculate the average total cost of parts (sum of Unit_price × Quantity across all **PartsOrderLine** records, divided by Total_VEHICLES) and display as Avg_parts_cost.
 - If Avg_parts_quantity >= 5 or Avg_parts_cost >= 500, highlight the row in red.
 - Sort the results by Total_VEHICLES descending, then by Avg_purchase_price ascending.
 - Display all rows in the report table.

```

SELECT
  COALESCE(
    CONCAT(ic.first_name, ' ', ic.last_name),

```

```

        bc.business_name
    ) AS seller_name,
    COUNT(DISTINCT p.vin) AS total_vehicles,
    ROUND(AVG(p.purchase_price), 2) AS avg_purchase_price,
    ROUND(COALESCE(SUM(vehicle_parts.total_qty), 0) / COUNT(DISTINCT
p.vin), 2) AS avg_parts_quantity,
    ROUND(COALESCE(SUM(vehicle_parts.total_cost), 0) / COUNT(DISTINCT
p.vin), 2) AS avg_parts_cost,
    CASE
        WHEN COALESCE(SUM(vehicle_parts.total_qty), 0) / COUNT(DISTINCT
p.vin) >= 5
            OR COALESCE(SUM(vehicle_parts.total_cost), 0) /
COUNT(DISTINCT p.vin) >= 500
        THEN 1 ELSE 0
    END AS highlight
FROM Customer c
LEFT JOIN IndividualCustomer ic ON c.customer_id = ic.customer_id
LEFT JOIN BusinessCustomer bc ON c.customer_id = bc.customer_id
JOIN Purchase p ON c.customer_id = p.seller_identifier
LEFT JOIN (
    SELECT po.vin,
        SUM(pol.quantity) AS total_qty,
        SUM(pol.unit_price * pol.quantity) AS total_cost
    FROM PartsOrder po
    JOIN PartsOrderLine pol ON po.parts_order_number =
pol.parts_order_number
    GROUP BY po.vin
) vehicle_parts ON p.vin = vehicle_parts.vin
GROUP BY c.customer_id, seller_name
ORDER BY total_vehicles DESC, avg_purchase_price ASC;

```

- ❖ On click **Back to Reports** button: Return to the **Reports Selection** form.

1.11 Generate Avg Time in Inventory Report

Abstract Code + SQL

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple using the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Average Time in Inventory Report** form by selecting the Average Time in Inventory option on the **Reports Selection** form and clicking the **View Report** button.

- ❖ Run the **Generate Average Time in Inventory Report** task as follows:
 - Retrieve all Vehicle types from **VehicleType**. For each type, find all sold Vehicles by joining **Vehicle**, **Purchase**, and **Sale** on vin.
 - Calculate the average days in inventory as: DATEDIFF(sale_date, purchase_date) + 1. The +1 ensures a Vehicle purchased and sold on the same day counts as 1 day, not 0.
 - **VehicleType** is the anchor of the query with LEFT JOIN to **Vehicle**, **Purchase**, and **Sale**. This ensures all Vehicle types appear in the output. **VehicleTypes** with no sold Vehicles produce NULL values, which CASE turns into "N/A".
 - Results are sorted by type_name in ascending alphabetical order.
 - Execute the following single query to produce the complete report data:

```
SELECT VehicleType.type_name,
CASE
  WHEN COUNT(Sale.vin) = 0 THEN 'N/A'
  ELSE CAST(
ROUND(AVG(DATEDIFF(Sale.sale_date, Purchase.purchase_date) + 1), 0)
  AS CHAR)
END AS avg_days_in_inventory
FROM VehicleType
LEFT JOIN Vehicle
  ON Vehicle.vehicle_type_name = VehicleType.type_name
LEFT JOIN Purchase
  ON Purchase.vin = Vehicle.vin
LEFT JOIN Sale
  ON Sale.vin = Vehicle.vin
GROUP BY VehicleType.type_name
ORDER BY VehicleType.type_name ASC;
```

- Note that the above query uses MySQL syntax. DATEDIFF() gets the number of days between two DATE values (see documentation at https://dev.mysql.com/doc/refman/8.4/en/date-and-time-functions.html#function_datediff). The CAST(... AS CHAR) lets the 'N/A' string and the numeric average days coexist in one column.
- Note that the query rounds the average number of days to the nearest whole day using ROUND().
- Display the result as a table on the **Average Time in Inventory Report** form with the following columns, in alphabetical order of type_name:
 - Vehicle Type: the type_name from **VehicleType**.
 - Average Days in Inventory
- On click **Back to Reports** button: Return to the **Reports Selection** form.

1.12 Generate Price Per Condition Report

Abstract Code + SQL

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple using the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Generate Price per Condition Report** form associated with the **Generate Price per Condition Report** task by selecting *Price per Condition* on the **Reports Selection** form and clicking the **View Report** button. **Reports Selection** can be accessed from **Vehicle Search/Result Form**.
- ❖ Run the **Generate Price per Condition Report** task as follows:
 - Display a matrix report on the **Price Per Condition Report** form with 5 rows (one per condition) and 9 columns (Condition plus one column per Vehicle type).
 - Rows are the five conditions in descending quality order: Excellent, Very Good, Good, Fair, Rough.
 - Columns are the eight Vehicle types from **VehicleType** in alphabetical order: Convertible, Coupe, Minivan, Other, Sedan, SUV, Truck, Van.
 - Each cell shows the average purchase_price from **Purchase** for all Vehicles in **Vehicle** matching that condition and Vehicle type. If no Vehicles match a condition/type pair, display "\$0".
 - The five condition rows are guaranteed by a derived table (inline UNION ALL query) since condition is an attribute of Vehicle, not a standalone table; conditions with no purchased Vehicles would otherwise be absent from the result.
 - Each Vehicle type column uses conditional aggregation: AVG(CASE WHEN ... THEN purchase_price END). The CASE returns the price for matching types and NULL otherwise; AVG ignores NULLs, so each column independently averages only its type's prices. COALESCE converts NULL results (no matching Vehicles) to "\$0".
 - The custom condition ordering is achieved via a CASE expression in ORDER BY that assigns a sort key (1–5) to each condition.
 - The exact query is as follows:

```
SELECT Conditions.`condition`,
COALESCE(CONCAT('$', ROUND(AVG(CASE WHEN Vehicle.vehicle_type_name =
'Convertible' THEN Purchase.purchase_price END), 2)), '$0') AS
Convertible,
COALESCE(CONCAT('$', ROUND(AVG(CASE WHEN Vehicle.vehicle_type_name =
'Coupe' THEN Purchase.purchase_price END), 2)), '$0') AS Coupe,
COALESCE(CONCAT('$', ROUND(AVG(CASE WHEN Vehicle.vehicle_type_name =
'Minivan' THEN Purchase.purchase_price END), 2)), '$
0') AS Minivan,
COALESCE(CONCAT('$', ROUND(AVG(CASE WHEN Vehicle.vehicle_type_name =
'Other' THEN Purchase.purchase_price END), 2)), '$0') AS Other,
COALESCE(CONCAT('$', ROUND(AVG(CASE WHEN Vehicle.vehicle_type_name =
'Sedan' THEN Purchase.purchase_price END), 2)), '$0') AS Sedan,
```



```

COALESCE(CONCAT('$', ROUND(AVG(CASE WHEN Vehicle.vehicle_type_name =
'SUV' THEN Purchase.purchase_price END), 2)), '$0') AS SUV,
COALESCE(CONCAT('$', ROUND(AVG(CASE WHEN Vehicle.vehicle_type_name =
'Truck' THEN Purchase.purchase_price END), 2)), '$0') AS Truck,
COALESCE(CONCAT('$', ROUND(AVG(CASE WHEN Vehicle.vehicle_type_name =
'Van' THEN Purchase.purchase_price END), 2)), '$0') AS Van
FROM ( SELECT 'Excellent' AS `condition` UNION ALL SELECT 'Very Good'
UNION ALL SELECT 'Good' UNION ALL SELECT 'Fair' UNION ALL SELECT 'Rough'
) AS Conditions
LEFT JOIN Vehicle ON Vehicle.`condition` = Conditions.`condition`
LEFT JOIN Purchase ON Purchase.vin = Vehicle.vin
GROUP BY Conditions.`condition`
ORDER BY CASE Conditions.`condition` WHEN 'Excellent' THEN 1 WHEN 'Very
Good' THEN 2 WHEN 'Good' THEN 3 WHEN 'Fair' THEN 4 WHEN 'Rough' THEN 5
END;

```

- Display the result as a matrix table on the **Price Per Condition Report** form.
- ❖ On click **Back to Reports** button: Return to the **Reports Selection** form.

1.13 Generate Parts Statistics Report

Abstract Code + SQL

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple by clicking the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigated to the **Parts Statistics Report** form associated with the **Parts Statistics Report** task by selecting Parts Statistics on the **Reports Selection** form and clicking the **View Report** button. **Reports Selection** can be accessed from **Vehicle Search** form.
- ❖ Run the **Generate Parts Statistics Report** task using the HTTP Session/Cookie associated to the user as follows:
 - Find all **Vendor** records and retrieve the associated **PartsOrder** records via the FULFILLS relationship and **PartsOrderLine** records via the EXISTS_IN relationship using **Vendor.vendor_name** identifying property as classifier.
 - For each **Vendor.vendor_name**:
 - Find the total quantity of parts coming from a particular **Vendor.vendor_name**, using **PartsOrder.parts_order_number** and **PartsOrder.order_ordinal** to identify the parts linked to a particular **Vendor.Vendor_name** and adding **PartsOrderLine.quantity** for all those items.
 - Find the total dollar amount spent on parts coming from a particular **Vendor.vendor_name**, using **PartsOrder.parts_order_number** and **PartsOrder.order_ordinal** to identify the parts linked to a particular

- Display `Vendor.vendor_name` ordered in alphabetical order together with the number of parts coming from that vendor and the total dollar amount spent on parts.

- ❖ Once the user finishes reading the report, it will be possible to go back to **Reports Selection** using the ***Back to Reports*** button.
- ❖ From **Reports Selection** the user is allowed to go back to Vehicle Search form by clicking on ***Back to Vehicle Search*** button.

```
--reppport
SELECT Vendor.vendor_name,
       SUM(PartsOrderLine.quantity) AS num_pieces_vendor,
       SUM(PartsOrderLine.quantity * PartsOrderLine.unit_price) AS
total_dollar_vendor
FROM PartsOrder
INNER JOIN PartsOrderLine ON PartsOrder.parts_order_number =
PartsOrderLine.parts_order_number
INNER JOIN Vendor ON Vendor.vendor_name = PartsOrder.vendor_name
GROUP BY Vendor.vendor_name
ORDER BY Vendor.vendor_name ASC;
```

1.14 Generate Monthly Sales/Drilldown Report

Abstract Code + SQL

- ❖ The operating manager, or the owner, successfully authenticates using username and password tuple by clicking the Login button in the **Vehicle Search/Result Form**.
- ❖ User navigates to the **Monthly Sales/Drilldown Report** form associated with the **Generate Monthly Sales/Drilldown Report** task by selecting Monthly Sales/Drilldown on the **Reports Selection** form and clicking the **View Report** button. **Reports Selection** can be accessed from **Vehicle Search/Result Form**.
- ❖ Run the Generate Monthly Sales/Drilldown Report task as follows:
 - Query all sold Vehicles by joining **Sale**, **Purchase**, and parts cost data. Group results by year and month of sale_date.
 - For each period, compute:
 - Total Vehicles sold: count of **Sale** records in the period.
 - Gross sales income: sum of calculated sale prices: $1.25 \times \text{purchase_price} + 1.10 \times \text{total_parts_cost}$.
 - Net income: sum of profit per Vehicle: $0.25 \times \text{purchase_price} + 0.10 \times \text{total_parts_cost}$.

- Parts cost per Vehicle is pre-aggregated in a CTE (**PartsCost**) to prevent inflating row counts when a **Vehicle** has multiple **PartsOrderLine** records. Vehicles with no parts use COALESCE(..., 0) to default to zero.
- Results are sorted by year and month descending (most recent first). Periods with no **Sale** get excluded.
- Run the following query to produce the complete report:

```
WITH PartsCost AS (
SELECT PartsOrder.vin, SUM(PartsOrderLine.unit_price *
PartsOrderLine.quantity) AS total_parts_cost
FROM PartsOrder INNER JOIN PartsOrderLine ON
PartsOrderLine.parts_order_number = PartsOrder.parts_order_number
GROUP BY PartsOrder.vin )
SELECT
YEAR(Sale.sale_date) AS sale_year,
MONTH(Sale.sale_date) AS sale_month,
COUNT(Sale.vin) AS total_VEHICLES_sold,
SUM( 1.25 * Purchase.purchase_price + 1.10 *
COALESCE(PartsCost.total_parts_cost, 0) ) AS gross_sales_income,
SUM( 0.25 * Purchase.purchase_price + 0.10 *
COALESCE(PartsCost.total_parts_cost, 0) ) AS net_income
FROM Sale INNER JOIN Purchase ON Purchase.vin = Sale.vin LEFT JOIN
PartsCost ON PartsCost.vin = Sale.vin
GROUP BY YEAR(Sale.sale_date), MONTH(Sale.sale_date)
ORDER BY sale_year DESC, sale_month DESC;
```

- Display the result set as a table on the Monthly Sales Report form with columns: Year, Month, Number of Vehicles Sold, Gross Sales Income, and Total Net Income.
 - Each row includes a **Drilldown** button that passes the selected year and month to the drilldown report.
- ❖ When the user presses the **Drilldown** button, the following operations are performed:
- For the selected period, find all **Sale** records and group by SalesAgent. Retrieve the agent's name by joining SalesAgent to StaffUser.
 - For each sales agent, compute the number of Vehicles sold and total sales amount.
 - Results are sorted by Vehicles sold descending, then total sales descending.
 - The drilldown report data is fully created using the following SQL query:

```
WITH PartsCost AS (
SELECT PartsOrder.vin, SUM(PartsOrderLine.unit_price *
PartsOrderLine.quantity) AS total_parts_cost
```

```

FROM PartsOrder INNER JOIN PartsOrderLine ON
PartsOrderLine.parts_order_number = PartsOrder.parts_order_number
GROUP BY PartsOrder.vin )
SELECT StaffUser.first_name, StaffUser.last_name, COUNT(Sale.vin) AS
Vehicles_sold, SUM( 1.25 * Purchase.purchase_price + 1.10 *
COALESCE(PartsCost.total_parts_cost, 0) ) AS total_sales
FROM Sale INNER JOIN Purchase ON Purchase.vin = Sale.vin INNER JOIN
StaffUser ON StaffUser.username = Sale.sales_agent_username LEFT JOIN
PartsCost ON PartsCost.vin = Sale.vin
WHERE YEAR(Sale.sale_date) = '$SelectedYear' AND MONTH(Sale.sale_date) =
'$SelectedMonth'
GROUP BY StaffUser.username, StaffUser.first_name, StaffUser.last_name
ORDER BY Vehicles_sold DESC, total_sales DESC;

```

- ❖ On click ***Back to Reports*** button from **Drilldown** form: go back to the **Monthly Sales/Drilldown Report**.
- ❖ On click ***Back to Reports*** button from **Monthly Sales Summary** form: go back to **Reports Selection**.
- ❖ On click ***Back to Vehicle Search*** button from **Reports Selection**: go back to **Vehicle Search/Result Form** form.